

2026 编译实验优化文章

针对 MIPS 的结构优化

不知道大家在使用 LLVM 作为中间代码时，会不会感受到这样的问题，那就是 LLVM 的语法结构过于死板，且有的时候和我们的直觉背道而驰。例如定义一个变量，在 LLVM 中需要先申请一片空间，返回指向空间的指针，再根据指针将初始值 store 进空间中！为数组申请空间的时候，返回的竟然是指向数组的指针！如果将指向数组的指针作为函数参数传递的话，参数的类型竟然是 Decay 之后的指向指针的指针！比较的结果和分支跳转的条件必须是 i1 类型，与 i32 类型之间必须严格区分！这些设计虽然保证了作为中间代码的严谨性，但也极大地限制了我们生成代码的性能，事实上我们很多时候都在做无用功。

于是，我们可以针对 MIPS 的相关特性，对我们的中间代码进行一些巧妙的优化，充分解决 LLVM 脱 XX 放 X 的问题：

优化变量的存储形式

在前面的吐槽中我们提到，原本的 LLVM 在定义变量时过于麻烦，我们希望能够用一种更符合直觉，且更容易生成 MIPS 指令的方式存储变量。

对于非数组的局部变量，我们不再为其分配空间，而是直接将其存入 LLVM 寄存器中。例如下面的语句：

```
; int n = 5
%1 = alloca i32
store i32 5, i32* %1
; int m = n
%2 = load i32, i32* %1
%3 = alloca i32
store i32 %2, i32* %3
```

经过优化之后，变为如下语句：

```
; int n = 5
%1 = 5
; int m = n
%2 = %1
```

对于数组的局部变量，我们仍然为其分配空间，但我们返回的不再是指向数组的指针，而是和 C 语言一样指向数组首地址的指针。例如下面的语句：

```
; int n[3] = {1, 2, 3}
%1 = alloca [3 x i32]
%2 = getelementptr inbounds [3 x i32], [3 x i32]* %1, i32 0, i32 0
store i32 1, i32* %2
%3 = getelementptr inbounds i32, i32* %2, i32 1
store i32 2, i32* %3
%4 = getelementptr inbounds i32, i32* %3, i32 1
store i32 3, i32* %4
; int m = n[0]
%5 = alloca i32
%6 = getelementptr inbounds [3 x i32], [3 x i32]* %1, i32 0, i32 0
%7 = load i32, i32* %6
store i32 %7, i32* %5
```

经过优化之后，变为如下语句：

```
; int n[3] = {1, 2, 3}
%1 = alloca [3 x i32]
store i32 1, i32* %1
%2 = getelementptr inbounds i32, i32* %1, i32 1
store i32 2, i32* %2
%3 = getelementptr inbounds i32, i32* %2, i32 1
store i32 3, i32* %3
; int m = n[0]
%4 = load i32, i32* %1
%5 = %4
```

全局变量同理，例如我们在调用 LLVM 全局变量 @a 时，若 @a 是非数组变量，则其代表的就是该变量的值；若 @a 是数组变量，则其代表的就是指向数组首地址的指针。

经过这样的改变，我们在将其翻译为 MIPS 指令的时候，同时也需要作出相应的调整。

对于局部变量，我们在定义新变量的时候使用了我们自创的赋值语句 LLVMMove，例如 %1 = 5
%2 = %1，这实际上就对应着 MIPS 中的 move 伪指令。

对于全局变量，我们在使用 MIPS 中 .data 段的 label 时，需要注意我们需要读取的是 label 对应的内容还是地址。对于非数组的全局变量，我们需要读取的是 label 对应的内容；而对于数组的全局变量，

我们需要读取的是 label 对应的地址。这就造成了一定的混乱，在读取全局变量之前，我还需要先确定它到底是不是数组，还是有一定困难的。

为了解决这个问题，我们可以改为使用下面这种方式定义数组：

```
a: .word a.data
a.data: .word 0, 1, 2, 3, 4, 5, 6, 7, 8, 9    #@a = dso_local global [10 x i32] [i32 0, i32 1, i32 2, i32 3, i32 4, i32 5, i32 6, i32 7, i32 8, i32 9]
```

通过这种方式，我们将全局数组 @a 的值存入了名为 a.data 的 MIPS 标签中，又将 a.data 标签的地址存入了名为 a 的 MIPS 标签中。这样，我们在读取 a 的内容时，实际上读取到的是 a.data 的地址，正好解决了我们的问题！

优化数组赋初始值

在定义数组局部变量的时候，我们在申请空间之后，需要将要赋的初始值一个一个写入内存中，普遍会带来大量的内存读写操作。针对 MIPS 的特性，我们可以对生成的中间代码进行如下优化：

我们在 MIPS 中申请空间时，初始值默认就是 0，所以当初始值为 0 的时候，我们不需要对其进行任何操作。

当然这也会带来一些问题，我们原本为数组赋初始值的实现方案中，指向每个元素的指针是依次递增的。如果我们跳过为数组中的某个元素赋初始值，就会出现一次没有任何意义的递增。为了解决这个问题，我们可以将指针的计算由依次递增改为绝对偏移，这样就解决了上述问题。

例如下面的语句：

```
; int n[5] = {0, 1, 0, 3}
%1 = alloca [5 x i32]
store i32 0, i32* %1
%2 = getelementptr inbounds i32, i32* %1, i32 1
store i32 1, i32* %2
%3 = getelementptr inbounds i32, i32* %2, i32 1
store i32 0, i32* %3
%4 = getelementptr inbounds i32, i32* %3, i32 1
store i32 3, i32* %4
```

经过优化之后，变为如下语句：

```

; int n[5] = {0, 1, 0, 3}
%1 = alloca [5 x i32]
%2 = getelementptr inbounds i32, i32* %1, i32 1
store i32 1, i32* %2
%3 = getelementptr inbounds i32, i32* %1, i32 3
store i32 3, i32* %3

```

优化判断条件

在 LLVM 中，`i1` 类型和 `i32` 类型之间有着严格的区别，为了保证生成的中间代码的正确性，优化前采用了非常保守的写法，即每当通过 `==` `!=` `<` `>` `<=` `>=` 运算得到一个 `i1` 类型变量时，都立刻使用 `zext` 指令将其无符号扩展为 `i32` 类型；在条件表达式最后，再将结果与 0 进行比较，得到 `i1` 类型的值之后，再使用 `br` 指令进行条件跳转。

然而，在 MIPS 中并没有 `i1` 类型和 `i32` 类型的区别，所有的值都是 32 位，所以我们并不需要在两种类型之间来回转换，于是我们可以删除 LLVM 中的 `zext` 指令，以及条件表达式最后与 0 比较的部分。例如下面的语句：

```

; if (m > n)
%3 = icmp sge i32 %1, %2
%4 = zext i1 %3 to i32
%5 = icmp ne i32 %4, 0
br i1 %5, label %6, label %7

```

经过优化之后，变为如下语句：

```

; if (m > n)
%3 = icmp sge i32 %1, %2
br i1 %3, label %4, label %5

```

（当然了，优化后的 `%3` 实际上并不是 `i1` 类型，优化后已经再也没有 `i1` 类型了）

优化跳转指令

在 LLVM 中，函数的任何一个基本块都必须以 `br` 指令或 `ret` 指令结尾，而 MIPS 中则不然。在 MIPS 中，如果 `label` 的前一条指令不是跳转指令，程序运行时会直接步入 `label` 中。于是我们可以针对 LLVM 中的 `br` 指令进行如下优化：

如果当前指令是 `br` 指令，则检查其下一条指令是否是 LLVM `label`。如果是，则检查该 `label` 与 `br` 指令要跳转到的 `label` 是否相同。

这里有两种情况，如果 `br` 指令是非条件跳转指令，那么问题就比较简单了，因为非条件跳转指令只有一个跳转 `label`。如果上述两个 `label` 相同，则不输出任何指令；否则输出 `j` 指令。

如果 `br` 指令是条件跳转指令的话，问题就会变得更复杂一些。如果条件不成立时跳转到的 `label` 和接下来的 `label` 相同，那么我们依然只需要省略 `j` 指令即可；但如果条件成立时跳转到的 `label` 和接下来的 `label` 相同，那么我们就需要将整条指令取反，将 `bnez` 指令变为 `beqz` 指令，这样就可以交换两个 `label`，使条件成立时跳转到的 `label` 变为条件不成立时跳转到的 `label`。例如下面的语句：

```
# br i1 %3, label %4, label %5
bnez $t0, label_4
j label_5
label_4:
    .....
label_5:
    .....
```

经过优化之后，变为如下语句：

```
# br i1 %3, label %4, label %5
beqz $t0, label_5
label_4:
    .....
label_5:
    .....
```

全局寄存器分配

在我看来，全局寄存器分配是一个非常重要，但也非常困难的优化环节。如果想实现标准的全局寄存器分配，需要经过划分基本块、活跃变量分析、构建冲突图、图着色分配寄存器等多个环节，每个环节的实现难度都相当巨大。接下来我将依次介绍全局寄存器分配中的每个环节：

启动全局寄存器分配

在理论课上我们已经学过，只有对局部变量才可以进行全局寄存器分配，而全局变量不可以进行全局寄存器分配。所以我们需要以每个函数为单位，为其中的所有 LLVM `label` 进行全局寄存器分配。

在我们进行目标代码生成时，当我们进入一个新的函数，就需要创建一个新的 `Optimizer` 类对象，向其中传入该函数中的所有 LLVM 指令，启动全局寄存器分配。

划分基本块

在划分基本块部分，我们需要完成以下 4 个容器的填写：

```
private ArrayList<String> blockNames;    // 基本块的标签名
private HashMap<String, ArrayList<LLVM>> blocks;    // 每个基本块中包含的 LLVM 指令
private HashMap<String, HashSet<String>> in2out;    // 指向每个基本块的基本块
private HashMap<String, HashSet<String>> out2in;    // 每个基本块指向的基本块
```

在这一部分中，我们遍历两趟整个函数。在第一趟，我们识别出基本块中所有的 LLVM label，将 label 的名称写入 blockNames 中，将 label 对应的基本块中的 LLVM 指令写入 blocks 中。这里需要注意的是，在 LLVM 中，函数的第一个基本块前面并没有 label，我们可以为 label 指定一个默认的初始值，例如 begin。

在第二趟，我们构建基本块之间的流图，主要需要观察基本块中的 br 指令。根据每个基本块的 br 指令，我们就能够得出基本块之间的跳转关系，从而构建出 in2out 和 out2in。

活跃变量分析

在活跃变量分析部分，我们需要完成以下 4 个容器的填写：

```
private HashMap<String, HashSet<String>> def;
private HashMap<String, HashSet<String>> use;
private HashMap<String, HashSet<String>> in;
private HashMap<String, HashSet<String>> out;
```

真是非常熟悉的 4 个容器！经验告诉我们，我们应该先求出 def 和 use，然后再不断迭代，求出 in 和 out。

首先我们先来求 def 和 use。这就需要我们遍历每个基本块中的每个 LLVM 指令，获取它们定义的变量和使用的变量的集合。于是我们在每个 LLVM 指令类中加入 getDef() 和 getUse() 方法。例如在 LLVM 指令 %3 = add i32 %1, %2 中，调用 getDef() 方法返回 String 类型的 %3，调用 getUse() 方法返回 HashSet<String> 类型的 {%1, %2}。

在获得了每条指令定义的变量和使用的变量的集合后，我们尝试将其加入当前基本块的 def 集合或 use 集合中。根据两个集合的定义，我们首先尝试加入使用的变量。我们遍历当前指令所有使用的变量，如果该变量不在 def 集合中，就将其加入 use 集合。接下来考察当前指令定义的变量（如果存在的话），如果该变量不在 use 集合中，就将其加入 def 集合。

接下来是进行迭代，求出 in 和 out。我们使用 do-while 循环，在每次循环开始前定义一个 over 变量，初始值为 true，一旦在循环过程中有某个 out 集合发生了改变，则将 over 置为 false，而

跳出循环的条件则是在循环末尾处，`over` 的值依然为 `true`。

在循环内部，我们倒序遍历所有的基本块。首先计算当前基本块的 `out` 集合，我们读取 `out2in` 容器中对应在基本块，将它们的 `in` 集合取并集即可。接下来使用 $in = use + (out - def)$ 公式，即可求出当前基本块的 `in` 集合。经过不断迭代，我们就实现了活跃变量分析。

构建冲突图

在构建冲突图部分，我们需要完成 1 个容器的填写：

```
private HashMap<String, HashSet<String>> crushMap;
```

`crushMap` 的 `key` 和 `value` 中的元素均为 LLVM label，也就是相互之间冲突的变量。那么，两个变量要怎样才算是冲突呢？我们采用最常见的一种定义方法，即在一个变量的定义点处，另外一个变量是活跃的。于是我们可以通过如下算法找到冲突变量：

我们倒序遍历每个基本块中的所有 LLVM 指令，计算在每条指令定义点处的活跃变量。那么如何计算每条指令定义点处的活跃变量呢？在遍历的最开始，我们首先复制一份当前基本块的 `out` 集合。在遍历每条指令时，如果当前指令存在定义点，如果该定义点在 `out` 集合中，就将其从集合中删去。此时 `out` 集合中的所有变量均是定义点处变量的冲突变量。接下来将该指令中使用的所有变量加入 `out` 集合中，继续向前遍历。

在得到了每个定义点处的活跃变量之后，我们就可以将其加入冲突图中了。需要注意的是，函数定义处的所有参数在同一个语句中均是定义点，它们两两之间均冲突，这里需要进行一下特判。

图着色分配寄存器

在图着色分配寄存器部分，我们需要完成 1 个容器的填写：

```
private label2regs = new HashMap<String, Register>();
```

在这个部分，我们需要复制一份冲突图，不断去掉冲突图中的节点，直到全部节点均被去掉为止。

在每次循环中，我们遍历所有节点，计算所有节点中的度的最小值和最大值，以及它们对应的节点。如果度的最小值小于可分配的寄存器，则为其对应的节点分配全局寄存器，并将该节点从图中移出；如果度的最小值大于等于可分配的寄存器，则将度的最大值对应的节点从图中移出。

在为节点分配全局寄存器时，我们需要检查该节点所有已分配寄存器的邻居分配到的寄存器，防止出现相邻两节点（冲突节点）被分配到同一个寄存器的情况。

这样，我们就完成了全局寄存器的分配，太不容易了！

其它简单的优化

常量折叠

常量折叠其实是相对非常简单的一种优化，实际上就是在遍历 AST，生成中间代码的过程中，将表达式中出现的常量直接计算出来。

例如在 `AddExp` 中，正常情况下，我们应该为被写入的变量申请一个新的 LLVM label，与被使用的两个变量的 LLVM label 一同构建 LLVM 的 `add` 指令，加入 LLVM 指令集合中。

然而，如果被使用的两个变量的 LLVM label 均是数字，我们就可以直接对这两个数字进行相加，将相加的结果作为被写入的变量的 LLVM label，整个过程不需要产生新的 LLVM 指令。

例如下面的语句：

```
; a = 3 + 5
%1 = 3
%2 = 5
%3 = add i32 %1, %2
%4 = %3
```

经过优化之后，变为如下语句：

```
; a = 3 + 5
%1 = 8
```

由于时间和精力有限，目前已经实现的优化就只有这么多。本来还打算尝试一下死代码消除和公共子表达式删除的，确实是来不及了，就这样吧。